

FLIPPED COURSE CAPSTONE PROJECT REPORT

EduPulse: Academic and Course Management System

A Modular Object-Oriented Software Application in Java

Student Name:	Maddala Jashwanth
Registration Number:	25BAI10796
Degree Program:	B.Tech Computer Science and Engineering (AI & DS)
Course Name:	Object-Oriented Programming (Java)
Course Code:	CSE2006
Submission Date:	September 2026
GitHub Repository:	https://github.com/jassu-dev/VITYARTI.git
Evaluation Component:	Flipped Learning / Build Your Own Project Evaluation

Submitted in partial fulfillment of the requirements for CSE2006. All code and documentation are original and adhere to academic honesty guidelines.

2. Introduction

Semester registration, course management, and academic evaluations are essential administrative functions in every university. In real campus environments, these activities involve multiple stakeholders: academic administrators who configure course catalogs and prerequisites; faculty members who monitor class rosters and record marks; and students who browse subjects, enroll, and track their grade point averages (GPA).

EduPulse is a modular, console-based academic management system written in Java SE for the **CSE2006 Object-Oriented Programming** course evaluation. The project applies core object-oriented principles (Encapsulation, Inheritance, Polymorphism, Abstraction) and standard GoF design patterns (Factory Method, Strategy, Singleton, Repository/DAO) to manage the entire student course registration and grade evaluation lifecycle.

3. Problem Statement

Manual course registration or systems that rely on uncoordinated spreadsheets suffer from several recurring problems:

- **Unverified Prerequisite Enrollments:** Students routinely attempt to register for advanced courses without clearing required introductory courses first.
- **Classroom Capacity Violations:** Popular subjects quickly exceed safe classroom limits if registrations are not synchronized against maximum seat capacities in real time.
- **Inconsistent Grading Scales:** Calculating cumulative GPAs requires weighting marks by individual course credits. Switching between a standard 4.0 scale and a 10-point scale manually creates calculation errors.
- **Scattered Academic Records:** Student records, instructor mark sheets, and enrollment files often get out of sync, making it difficult to generate accurate, unified transcripts.

4. Functional Requirements

The system is divided into five core functional modules:

Module	Key Responsibilities & Input/Output Flow
1. Authentication & Roles	Handles user login with SHA-256 encrypted passwords. Dispatches users to specific dashboards for Administrator, Faculty, or Student.
2. Course Catalog	Allows administrators to create courses, configure credits (1 to 6), set seat limits, assign faculty, and define prerequisite dependencies.
3. Enrollment Engine	Validates prerequisite completion, checks seat availability, enforces a maximum of 6 courses per student, prevents duplicate enrollments, and handles course drops.
4. Grading Engine	Enables faculty to submit marks (0 to 100). Automatically converts marks into letter grades using a selected strategy and recalculates cumulative GPA.
5. Transcripts & Analytics	Generates official academic transcripts for students and cohort performance reports (averages, highest/lowest marks, pass rate, grade counts) for instructors.

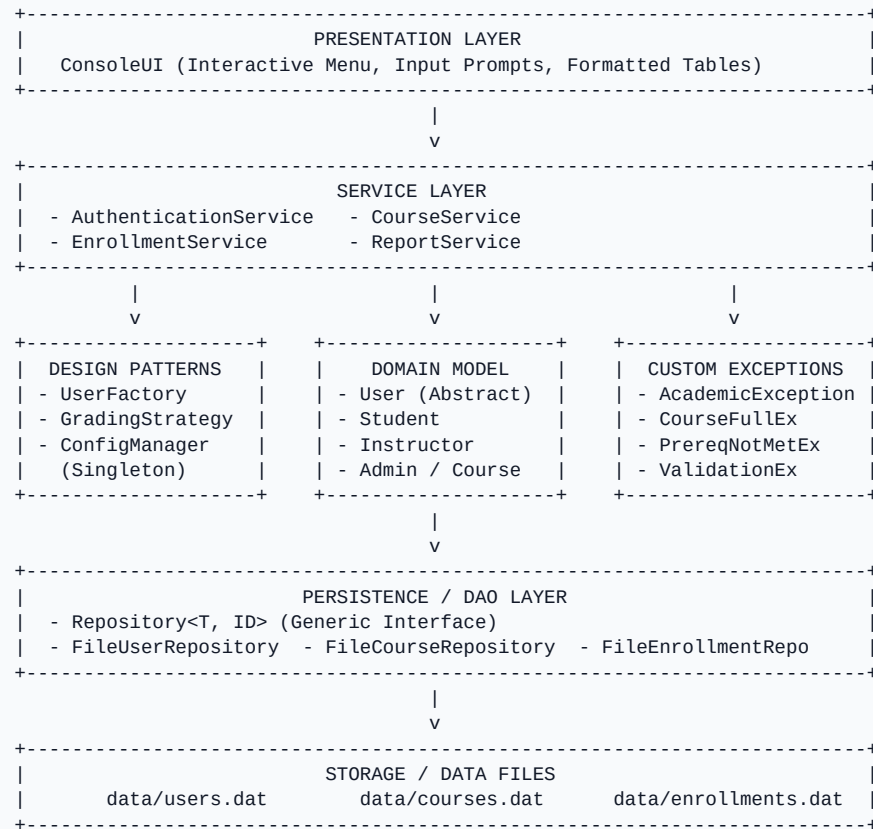
5. Non-Functional Requirements

- **Performance:** Fast in-memory lookups using ConcurrentHashMap ensure instantaneous catalog queries and enrollment checks.
- **Security:** User passwords are encrypted with SHA-256 before storage. Critical fields are kept private to protect internal state.

- **Usability:** The console interface features structured menus, cleanly aligned tables, and descriptive error messages.
- **Data Persistence:** Standard Java binary serialization flushes state to disk automatically, ensuring data survives program restarts.
- **Maintainability:** Clear separation into Model, Repository, Service, and UI layers makes the codebase straightforward to inspect and extend.

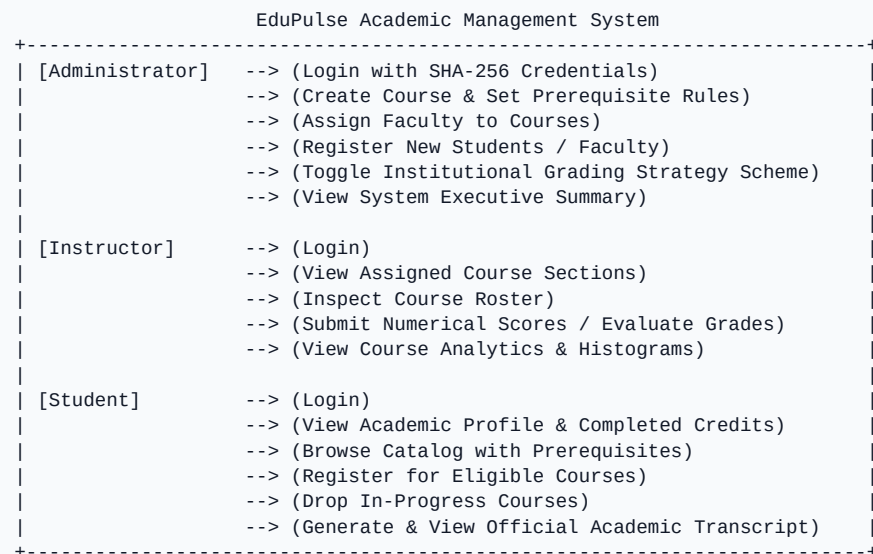
6. System Architecture

EduPulse is structured into a clean Four-Tier Architecture to separate user interaction from domain rules and file storage:

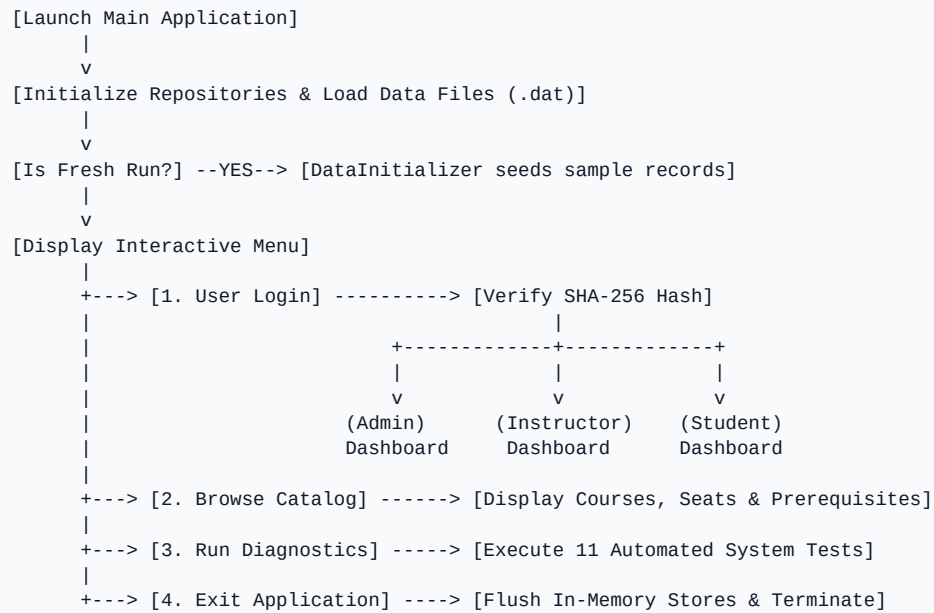


7. Design Diagrams

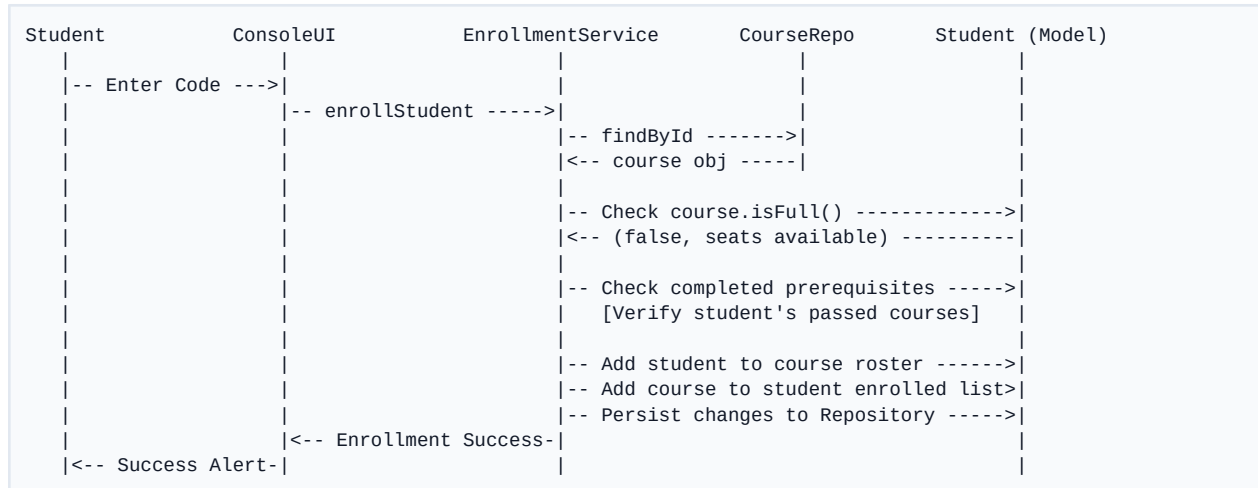
7.1 Use Case Diagram



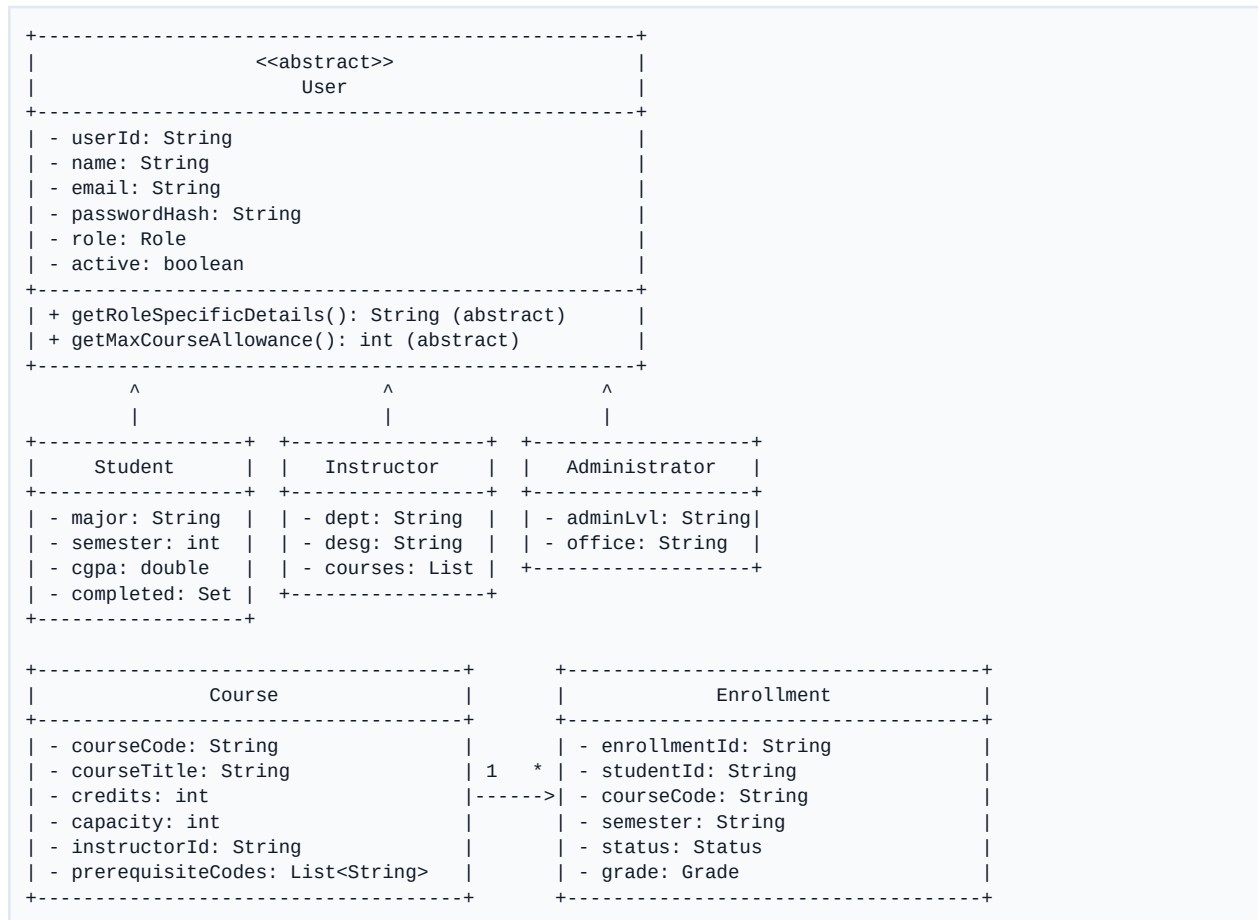
7.2 Process Flow / Workflow Diagram



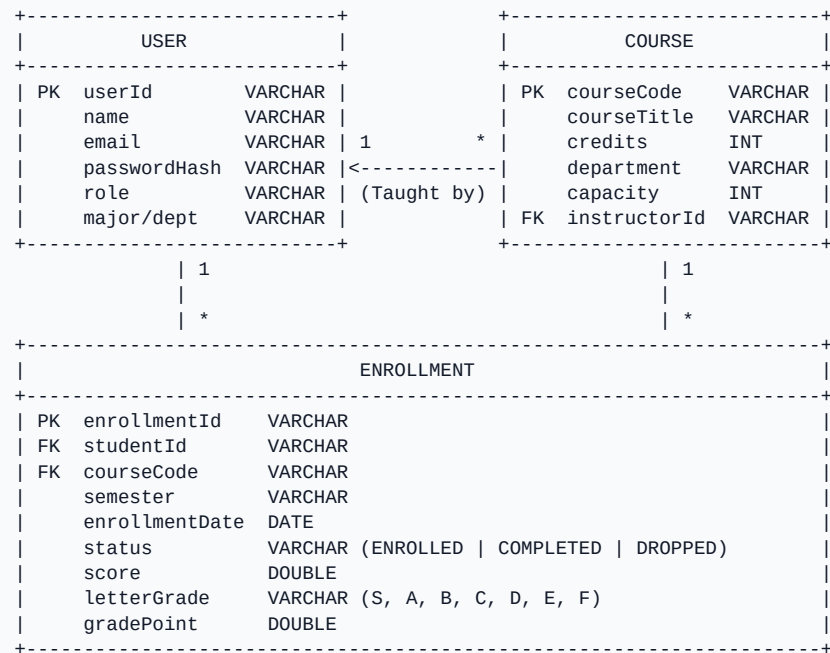
7.3 Sequence Diagram: Student Course Enrollment Flow



7.4 Class / Component Diagram



7.5 Entity-Relationship (ER) Storage Diagram



8. Design Decisions and Rationale

- **Layered Package Layout:** Dividing classes into model, repository, service, and UI ensures clear separation of concerns. The console interface can be replaced with a JavaFX or web UI without modifying service logic.
- **Strategy Pattern for Evaluations:** Decoupling grading algorithms into the `GradingStrategy` interface allows switching between 10-point and 4.0 scales at runtime without modifying service code.
- **Factory Method for User Creation:** `UserFactory` centralizes user construction, ensuring passwords undergo SHA-256 hashing uniformly upon registration.
- **Custom Checked Exceptions:** Specialized exceptions like `CourseFullException` and `PrerequisiteNotMetException` provide descriptive error messages to users rather than generic failures.
- **Java Serialization for Persistence:** Using `ObjectOutputStream` eliminates third-party database dependencies, making the project portable and easy to run on any standard JDK installation.

9. Implementation Details

EduPulse applies core Object-Oriented Programming concepts in a real-world scenario:

- **Encapsulation:** All fields in `User`, `Student`, `Course`, and `Enrollment` are private. Mutations occur through validated setters or domain methods. Collections are returned using `Collections.unmodifiableList()` to prevent external tampering.
- **Inheritance:** `Student`, `Instructor`, and `Administrator` all inherit common fields (`userId`, `name`, `email`, `passwordHash`, `role`) from the abstract `User` class.
- **Polymorphism:** Abstract methods `getRoleSpecificDetails()` and `getMaxCourseAllowance()` are dynamically dispatched based on the concrete user type.
- **Abstraction:** The service layer interacts with data stores solely through the generic `Repository` interface.

10. Execution Results and Output

10.1 Automated Test Runner Output (All 11 Tests Passed):

```
-----  
STARTING AUTOMATED UNIT & INTEGRATION TEST SUITE  
-----  
[PASS] UserFactory creates polymorphic subtype correctly  
[PASS] Polymorphic method getMaxCourseAllowance() returns role-specific capacity  
[PASS] SHA-256 password hash verifies matching credential  
[PASS] SHA-256 password hash rejects incorrect credential  
[PASS] StandardGradingStrategy correctly maps 92% to A+ (4.0)  
[PASS] HonorsGradingStrategy correctly maps 92% to S (10.0)  
[PASS] Prerequisite enforcement blocks enrollment when prerequisite course missing  
[PASS] Capacity constraint prevents enrollment beyond course capacity limit  
[PASS] Duplicate course registration rejected with ValidationException  
[PASS] Course drop frees seat in Course and updates enrollment status  
[PASS] Weighted GPA calculated accurately across multiple credit courses  
  
-----  
TEST EXECUTION SUMMARY: 11 / 11 PASSED (100.0% SUCCESS)  
-----
```

10.2 Sample Official Student Transcript:

OFFICIAL ACADEMIC TRANSCRIPT - VIT Bhopal / VITYarthi University						
Student Name : Aditya Sharma			Student ID : 23BCE0001			
Major : B.Tech Computer Science			Semester : 3			
Email : aditya.s@vityarthi.edu			Active Term: Fall 2026-27			
CODE	COURSE TITLE		CREDITS	SCORE	GRADE	STATUS
CSE2001	Object Oriented Programming...		4	94.5%	S	COMPLETED
CSE2002	Data Structures & Algorithms		4	N/A	In-Prog	ENROLLED
Total Attempted Credits: 8 Total Earned Credits: 4 Cumulative GPA: 10.00						
Academic Standing: Dean's Honor List / Distinction						

11. Testing Approach

The system was tested using an automated test suite (`SystemValidationTest.java`) with 11 targeted test cases:

- **Unit Tests:** Verified password hashing (`SecurityUtils`), user creation (`UserFactory`), and strategy conversions (`StandardGradingStrategy`, `HonorsGradingStrategy`).
- **Validation Tests:** Checked that missing prerequisites throw `PrerequisiteNotMetException` and full classes throw `CourseFullException`.
- **Lifecycle Tests:** Confirmed that dropping a course restores available seats in the course roster.
- **Calculation Tests:** Validated that weighted cumulative GPAs match manual mathematical calculations across courses with varying credit counts.

12. Challenges Faced

- **Handling Cyclic Prerequisites:** When administrators configure prerequisites, cycles could occur (Course A requires Course B, which requires Course A). Added cycle checks in `CourseService.addPrerequisite()` before saving.
- **Credit-Weighted GPA Derivation:** Since different subjects carry 3 or 4 credits, a simple average is inaccurate. Implemented weighted formula: $\text{sum}(\text{GradePoint} * \text{Credits}) / \text{sum}(\text{Credits})$.
- **Scanner Newline Trapping in Console:** Reading numbers with `scanner.nextInt()` left leftover newline characters in the buffer. Standardized on `scanner.nextLine()` with explicit integer parsing.

13. Learnings and Key Takeaways

- Applied core OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction) to a practical administrative system.
- Implemented software design patterns (Factory, Strategy, Singleton, DAO) cleanly to make code maintainable and extensible.
- Gained practical experience designing custom exception hierarchies and writing automated test cases in pure Java.

14. Future Enhancements

- Developing a desktop graphical interface using JavaFX or a web interface with Spring Boot.
- Connecting to a relational database like PostgreSQL or MySQL using JDBC/JPA.
- Adding automated email notifications when grades or announcements are published.

15. References

1. Oracle Java SE Documentation: <https://docs.oracle.com/en/java/>
2. Gamma, E. et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley.
3. Bloch, Joshua, *Effective Java* (3rd Edition), Addison-Wesley.
4. VIT Academic Regulations and Syllabus for Object-Oriented Programming (Java - CSE2006).